

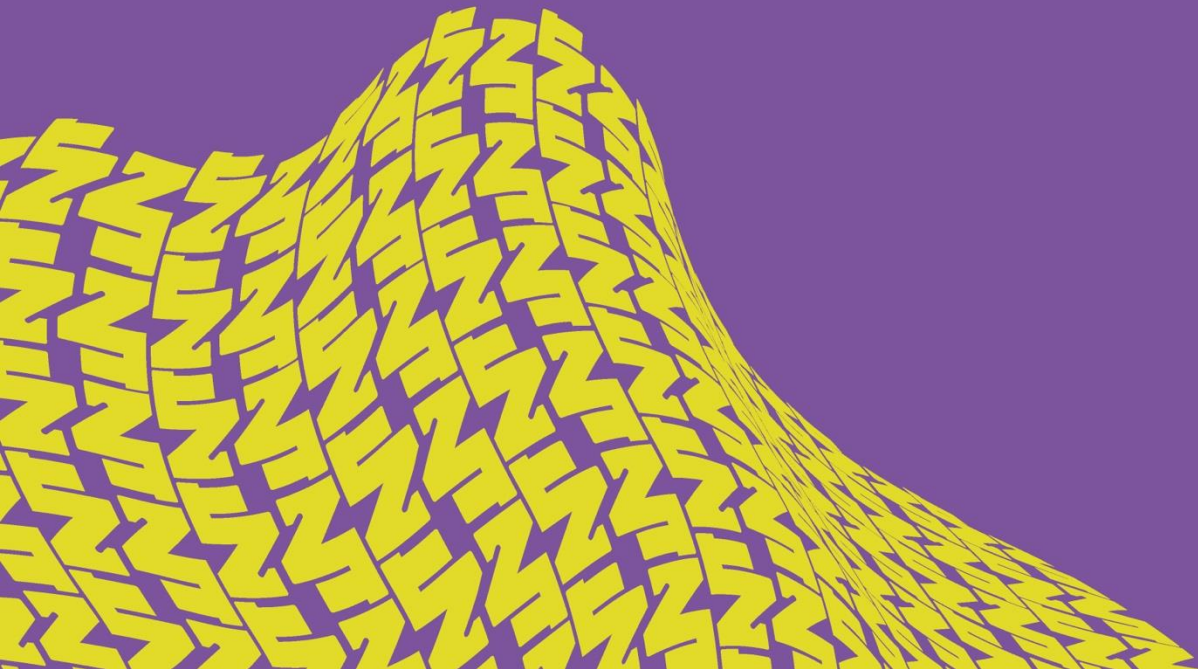


NT conference
21 – 23 September
2026



Data science in Azure with or without LLMs?

Tomaž Kaštrun, MVP



Intro😊duction😊

- Data scientist | BI Developer | Data Analyst
- SQL Server, SAS, R, Python, C#, SAP, SPSS
- 20+years experience MSSQL, DEV, BI, DM, DS
- Frequent community speaker, book author
- Avid coffee drinker & bicycle junkie



Session material:

https://github.com/tomaztk/Data_science_LLM



<http://tomaztsql.wordpress.com>



tomaz.kastrun@gmail.com



[@tomaz_tsql](https://twitter.com/tomaz_tsql)



[/in/tomaztsql](https://in.linkedin.com/in/tomaztsql)



<http://github.com/tomaztk>



<https://mvp.microsoft.com/PublicProfile/5002196>

Agenda

1. Spoiler first: what LLMs changed in data science – and what they didn't
2. Chronology 2018 → today: how Azure grew up alongside GenAI
3. Before → After, layer by layer: IDE, languages, visualizations, the DS journey, algorithms, explainability
4. New toys: Multi-Agent RL and Graph Neural Networks
5. War stories: hallucinated court cases, a very honest chatbot, one deleted database – and growing tech debt
6. Demo, scorecard, takeaways – and your questions (answered by a human)

Spoiler: New Tools, Same Job

What LLMs changed – the “how”

Code: from typing it to reviewing it

Interface: English joined Python and SQL as a query language

Speed: first drafts of code, charts and explanations in seconds, not days

Scope: agents now plan, run and self-correct whole experiments

What didn't change – the “why”

Garbage in, garbage out – now with better grammar

Statistics, validation and domain knowledge still decide whether an answer is right

Someone is still accountable for the decision – and it isn't the model

Data preparation is still most of the job. Sorry.

Watch for the purple “Still the same” section on every Before → After slide.



Chronology: 2018 → Today

Eight years, five hype cycles, and one CSV that is still dirty

2018–2021: Transformers Move In

Data science shift

Transformers take center stage; GPT-3 (2020) makes text generation the new party trick
Classical ML still runs the business; NLP tasks (classification, entities) move to pre-trained models like BERT
Less hand-crafted features, more fine-tuning

Azure's response

Nov 2021: Azure OpenAI Service preview – GPT-3 behind enterprise security, compliance and data privacy
Responsible AI guardrails built in from day one

Meanwhile, in the data team: still fixing date formats in final_v7_REALLY_final.csv

Sources: ODSC – Open Data Science · Microsoft Azure



2022: Alignment, MLOps & ChatGPT

Data science shift

Enterprises move from notebooks to production; MLOps (MLflow, Kubeflow, monitoring) goes mainstream
RLHF and supervised fine-tuning teach models to follow instructions – and hallucinate a bit less
Nov 2022: ChatGPT launches, and every manager suddenly has “an AI strategy”

Azure's response

Azure ML matures enterprise MLOps: model registries, drift detection, pipeline orchestration
The plumbing gets laid for what comes next

Meanwhile: “Can’t ChatGPT just do the forecast?” is asked in a meeting for the first time. Not the last.

Sources: ODSC – Open Data Science · Medium



2023: The Generative AI Explosion

Data science shift

GPT-4, Claude, Llama and open-source LLMs move from demo to product
New core skills: prompt engineering, RAG, vector databases
LangChain and LlamaIndex let LLMs call tools, query databases and write code

Azure's response

Jan 2023: Azure OpenAI Service becomes generally available
Vector search for RAG in Azure (Cognitive → AI) Search; Prompt Flow lands in Azure ML

Meanwhile: every job title gets "AI" appended. The SQL queries stay exactly the same.

Sources: ODSC – Open Data Science · arXiv · Microsoft



2024: Agents & Multimodality

Data science shift

“AI engineering” grows beyond MLOps: agents, autonomous systems, scalable deployment
Multimodal models (GPT-4o, Gemini) handle text, images, audio and tables in one pipeline
Multi-agent frameworks (AutoGen & co.) write code, clean data and draft reports from a single instruction

Azure’s response

Azure AI Studio goes GA, then becomes Azure AI Foundry – one place to build, fine-tune and deploy LLM apps
GPT-4o fine-tuning in the platform; DPO arrives as a lighter alternative to RLHF

Meanwhile: the agent writes 400 lines of pandas in 4 seconds. Code review takes 4 hours.

Sources: ODSC – Open Data Science · Taylor & Francis Online · Microsoft Learn



2025–26: Agentic DS & Foundry

Data science shift

The role shifts from builder to orchestrator of agents

Reasoning models (o1/o3, DeepSeek-R1) bring step-by-step reasoning to analytical tasks

Data agents run experiments, test hypotheses and self-correct – moving from papers into early production

Azure's response

Model catalog explodes: 1,900+ models by April 2025, 10,000+ today (OpenAI, Anthropic, Meta, Mistral, DeepSeek, Phi...)

Model Router picks a model per query; Foundry Agent Service orchestrates multi-step, multi-model workflows

Azure AI Foundry becomes Microsoft Foundry; the Assistants API retires in Aug 2026

Meanwhile: the platform got its third name in three years. The data still has the same quality issues.

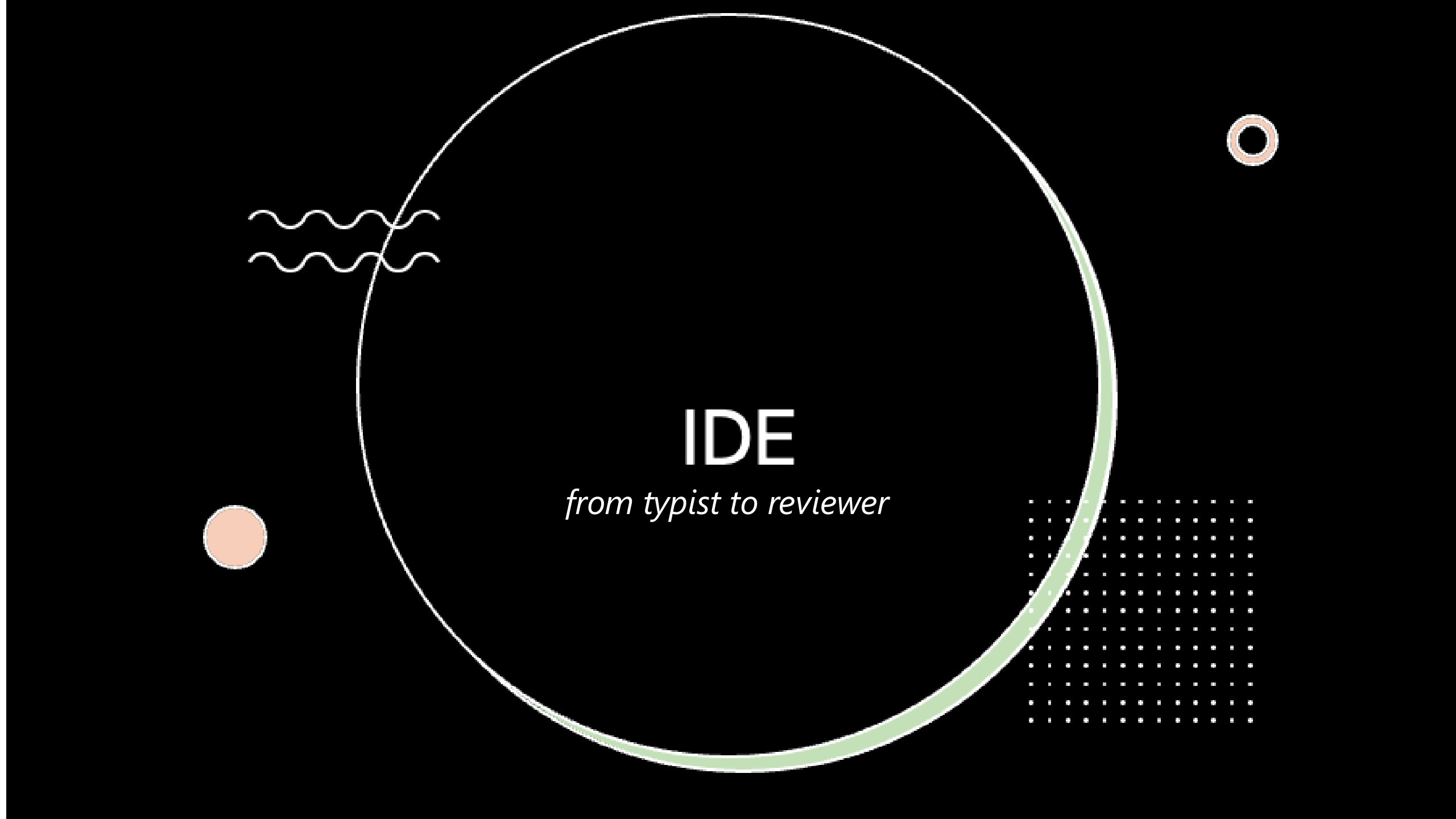
Sources: MSDynamicsWorld.com · arXiv · Microsoft Learn



The big picture

The throughline: manual pipelines → assisted workflows → autonomous agents.
Azure followed the same arc – from a single model API to a full AI orchestration platform (renaming it twice along the way).

YEAR	DATA SCIENCE CHANGE	AZURE RESPONSE
2021	GPT-3, pre-trained NLP models rise	Azure OpenAI Service preview (Nov)
2022	RLHF, ChatGPT, MLOps peak	MLOps tooling expansion in Azure ML
2023	RAG, LangChain, vector DBs, fine-tuning	Azure OpenAI GA, vector search, Prompt Flow
2024	Multi-agent systems, AI engineering, GPT-4o	AI Studio → Azure AI Foundry, GPT-4o & DPO fine-tuning
2025–26	Agentic DS, full-lifecycle automation	Microsoft Foundry: 10,000+ models, Agent Service, Model Router

An abstract graphic design on a black background. A large circle is centered, with a thin white outline and a thick green inner ring. To the left of the circle is a solid orange circle. Above the circle are two horizontal wavy lines. To the right of the circle is a small orange ring and a grid of small white dots. The text 'IDE' is centered within the circle, with the subtitle 'from typist to reviewer' below it.

IDE

from typist to reviewer

IDE: From Typist to Reviewer

Before – the IDE as a text editor with opinions

Jupyter, VS Code, PyCharm, Azure ML notebooks – every line typed by hand
Autocomplete knew syntax, not intent; boilerplate rewritten on every project
Debugging: one error, one Stack Overflow tab, one coffee at a time

After – AI-assisted, agentic IDE

GitHub Copilot in VS Code and Foundry generates functions, tests and whole notebooks from a comment
Agentic IDEs and CLI agents run the code, read the traceback and fix it in a loop; local LxMs for private work
Notebook assistants (Copilot in Fabric, Data Wrangler) suggest and explain cleaning steps

Still the same

You own every line that gets merged – “Copilot wrote it” is not a valid git blame
Readable, tested, version-controlled code is still the definition of done

IDE in practice: cleaning a sales dataframe

Before - typed by hand, line by line

```
def clean_sales(df):  
    df = df.dropna(subset=["amount"])  
    df["amount"] = df["amount"].astype(float)  
    df["date"] = pd.to_datetime(df["date"])  
    return df[df["amount"] > 0]
```

After - one prompt to Copilot / AI Foundry

```
# "Clean sales_df: drop missing amounts, cast amount  
# to float, parse dates, remove negative sales,  
# and add a test"  
→ generates the function above, a pytest test,  
   and a docstring - in seconds
```

**# Still your job: "remove negative sales" also removed every refund.
Great for revenue. Less great for the audit.**

Languages

English is the hottest new programming language

Languages: English Joins the Stack

Before – fluency in general-purpose languages

Python, R, SQL, Scala – deep fluency required for every task

Moving Python ↔ SQL ↔ Spark meant rewriting the same logic three times

The learning curve was the main barrier to entry

After – natural language as an interface layer

Natural language becomes a first-class “interface language” next to Python and SQL

NL2SQL and Copilot in Microsoft Fabric turn a question into a runnable query

Prompt design (and Prompt Flow in Foundry) becomes a core, testable skill

Still the same

SQL turned 50 and still isn't retiring – you need it to read what the AI wrote

Ambiguity is the real bug: “sales last year” meant three different things before LLMs, too

Languages in practice: querying sales data

Before - SQL written by hand

```
SELECT region, SUM(amount) AS total_sales
FROM sales
WHERE order_date >= '2026-01-01'
GROUP BY region
ORDER BY total_sales DESC;
```

After - natural language prompt

```
# "Show total sales by region since Jan 2026,
#  highest first"
→ Fabric Copilot / Azure OpenAI generates the
   SQL above directly against the semantic model
```

Still your job: is amount net or gross? Are returns in?

The AI doesn't know. Neither does the column name.

The background is a complex, abstract fractal pattern in shades of teal and blue. It features a dense, repeating pattern of fine, curved lines that create a sense of depth and movement, resembling a microscopic view of a mineral or a digital fractal generation.

Visualizations

from matplotlib to “make it pop”

Visualizations: From matplotlib to “Make It Pop”

Before – hand-built charts

Every chart hand-coded in matplotlib, seaborn or a BI tool

Choosing and styling the right chart type took real design effort

Every iteration meant rewriting the plotting code

After – natural language to chart

Describe the chart in plain language, get it back rendered

Copilot in Power BI drafts visuals, DAX measures and narrative summaries

Auto-visualization (LIDA-style) proposes the chart type from the data

Still the same

A misleading axis is still misleading – now produced faster and at scale

A pie chart with 14 slices is still a crime, whoever generated it

Visualizations in practice: monthly revenue trend

Before - matplotlib written line by line

```
fig, ax = plt.subplots(figsize=(8, 4))  
ax.plot(df["month"], df["revenue"], marker="o")  
ax.set_title("Monthly Revenue")  
ax.set_xlabel("Month"); ax.set_ylabel("Revenue")  
plt.xticks(rotation=45)
```

After - one prompt, same result

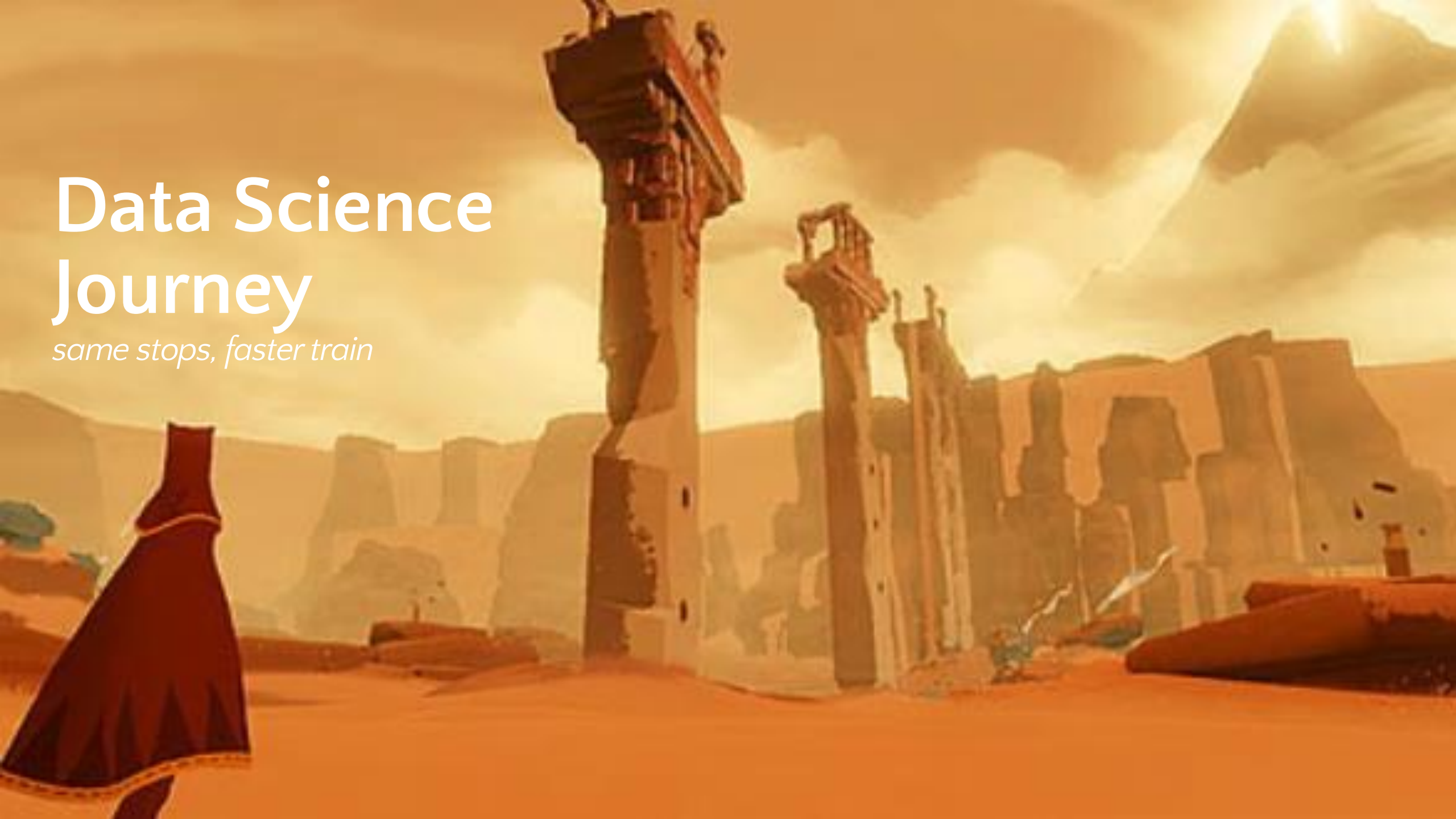
```
# "Plot monthly revenue as a line chart, label the  
# axes, and rotate the month labels"  
→ Copilot / Azure AI Foundry code interpreter  
   writes and runs the chart for you
```

Iterations 2-6: "make it pop", "more corporate", "less corporate"

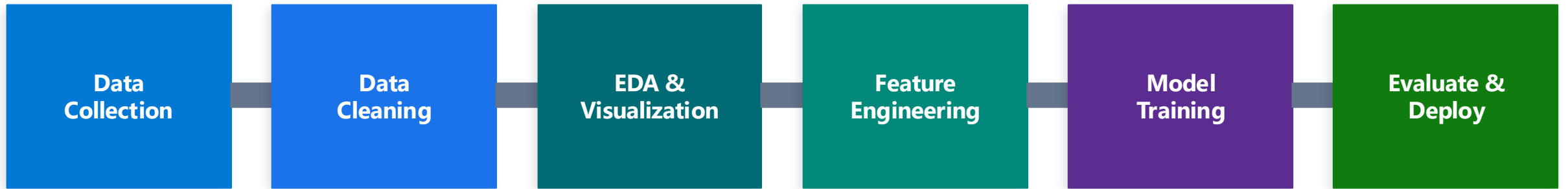
Still your job: check the axis isn't lying.

Data Science Journey

same stops, faster train

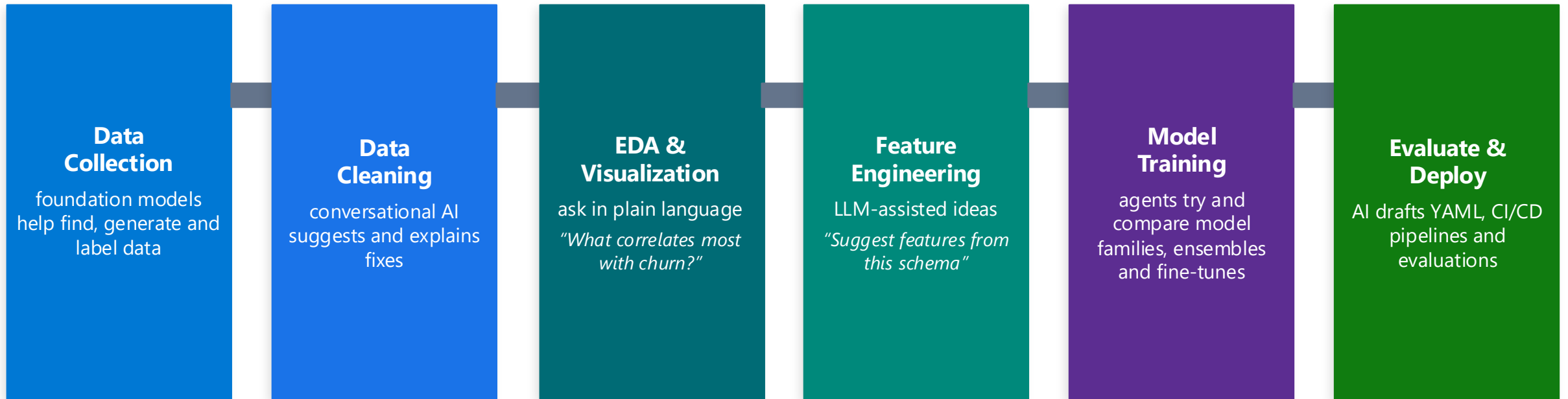


Data Science Journey – Pre-LLM



The classic time split: most of it finding and cleaning data, the rest modelling – and 100% explaining why it took so long.

Data Science Journey – After LLM



Same six stops, faster train. Most of the time still goes into the data – you just have a chatbot to complain to now.

NLP Tasks: From Research Project to API Call

Task effort comparison (accuracy still sold separately)

Task	Before	After LLM
Sentiment Analysis	Weeks	Minutes
Text Classification	2–4 weeks	Hours
Named Entity Recognition	Months	Day
Question Answering	Research project	API call
Summarization	Custom seq2seq	Prompt
Language Detection	fastText model	Zero-shot

Algorithms

gradient boosting sends its regards

Algorithm Landscape: Who Survived the GenAI Era?

Algorithm	Cat.	2025/6 Verdict	What changed
Linear / Logistic Regression	Classical	↑ Still baseline	Baseline, explainability, small data — LLMs can't beat it here
Decision Trees / Random Forest	Classical	↑ Still core	Tabular data king. LightGBM/XGBoost still dominate kaggle competitions
SVMs	Classical	→ Niche	Rare now. Kernel SVMs replaced by GBMs/neural nets for most tasks
LSTM / RNNs	Deep	↓ Mostly replaced	Transformers outperformed RNNs on all sequential tasks. Still used in edge devices.
CNNs	Deep	→ Image-specific	Still best for pure vision tasks. ViT challenging but CNNs remain efficient.
Transformers	Deep	↑↑ Dominant	The algorithm of the era. Attention mechanism powers all frontier LLMs.
Foundation Models (GPT, BERT...)	GenAI	★ Paradigm shift	Pre-train once, fine-tune or prompt for anything. Replaces per-task training.
RAG / Retrieval Models	GenAI	★ New pattern	Non-parametric knowledge — combine retrieval + generation. Azure AI Search.
Embedding Models	GenAI	↑↑ Critical infra	text-embedding-3-large powers search, RAG, clustering, dedup at scale.

Algorithms: From Grid Search to Agentic Experiments

Before – manual tuning and classic AutoML

- Algorithm choice and hyperparameters by hand, or grid/random-search AutoML
- New model family = new pipeline; one model, one run, one result at a time
- Feature engineering: manual, slow – and secretly the part that mattered most

After – agentic experimentation

- Agents propose, run and compare many models – and self-correct from the metrics
- Reasoning models (o1/o3, DeepSeek-R1) handle multi-step analytical reasoning
- New approaches (MARL, GNNs) extend *what* we model, not just how we tune it

Still the same

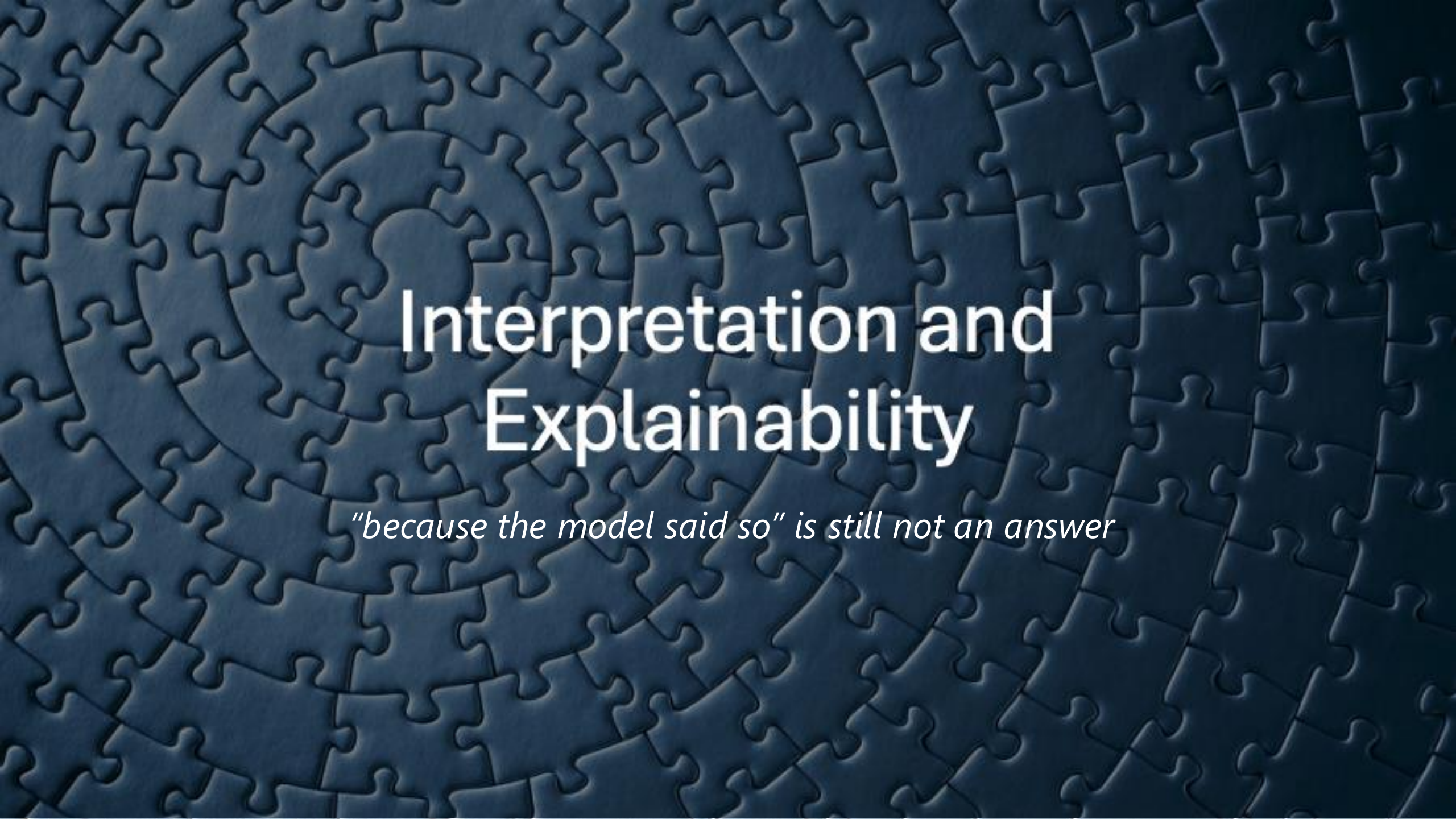
- On tabular data XGBoost/LightGBM still win most fights; logistic regression is still the baseline to beat
- Leakage, overfitting and the bias–variance trade-off did not read the GPT-4 paper

Algorithms in practice: choosing a model

```
# Before - manual grid search
from sklearn.model_selection import GridSearchCV
params = {"n_estimators": [100, 300], "max_depth": [4, 8]}
grid = GridSearchCV(RandomForestClassifier(), params, cv=5)
grid.fit(X_train, y_train)
best_model = grid.best_estimator_

# After - agent-orchestrated experimentation
# "Try Random Forest, XGBoost and a small GNN on
# this churn dataset, cross-validate, report best F1"
→ Foundry Agent Service runs the experiments,
   compares metrics, and returns the winning model

# Still your job: is F1 even the right metric? (Ask finance.)
# And did the agent leak the target into the features?
```



Interpretation and Explainability

"because the model said so" is still not an answer

Explainability: From SHAP Plots to Plain English

Before – manual interpretation

- SHAP / LIME plots generated and interpreted by the data scientist

- Translating plots into business language by hand – usually the night before the steering committee

- Explainability: a separate, often-skipped final step

After – LLM-narrated explainability

- LLMs turn SHAP/LIME output into plain-language explanations for any audience

- Agentic frameworks like XMODE expose every step of the analysis pipeline

- Explanations become continuous – part of every agent's output

Still the same

- Feature importance is not causation – an eloquent story about a correlation is still a correlation

- A fluent explanation can be wrong: check that the narrative matches the numbers

Evaluation: Old Metrics, New Metrics

Classical ML Evaluation
AUC-ROC / RMSE / F1
Confusion matrix
SHAP global importance
PDP / ICE plots
Fairness: demographic parity
Error analysis cohorts

GenAI / LLM Evaluation (new – and additional)	
Groundedness	Is answer supported by context? (RAG)
Relevance	Does answer address the question?
Coherence	Is the response logically consistent?
Fluency	Is the language natural and readable?
LLM-as-Judge	A model grades model outputs – who grades the grader?
Token cost / latency	New operational efficiency metrics

Upgraded Explainability Methods

XMODE – an LLM agent decomposes a natural-language question into an explicit workflow (SQL, analysis, plots), so every answer shows *how* it was produced

TalkToModel – a conversational interface to ask a model “why?”, “what if?” and “what would change this prediction?”

LLM-narrated SHAP & counterfactuals – classical explainers underneath, a plain-language story on top

LLM-as-judge + human review – for explaining and scoring the LLM’s own outputs

Still the same

An explanation is only as honest as the attribution method underneath it – the LLM is the narrator, not the witness

Explainability in practice: explaining a prediction

```
# Before - reading a SHAP plot by hand
import shap
explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_test)
shap.summary_plot(shap_values, X_test)
# → data scientist writes up the findings by hand

# After - LLM narrates the explanation
prompt = f"Explain in plain English why this customer"
        f" was predicted to churn: {shap_values[0]}"
response = client.chat.completions.create(
    model="gpt-4o", messages=[{"role": "user", "content": prompt}])
# → ready-to-share explanation, no manual write-up

# Still your job: verify the story matches the SHAP values -
# LLMs are excellent at confident narratives.
```

The background of the slide is a dark green field filled with a complex network of glowing green lines. These lines connect numerous blue circular nodes of varying sizes, creating a web-like structure that suggests interconnectedness and data flow. The nodes and lines are semi-transparent, allowing the dark background to show through.

Algorithms and New Approaches

MARL & GNNs: when your rows start talking to each other

Multi-Agent Reinforcement Learning (MARL)

What it is

Multiple learning agents share one environment, each optimizing its own reward while adapting to the others

Classical RL: one agent, one policy, one reward, a static world

Why it matters for data science now

Many business problems are multi-agent by nature – pricing, supply chain, resource allocation, multi-bot orchestration

From “optimize one model” to “optimize a system of interacting decision-makers”

Azure ML clusters and Foundry Agent Service can host and coordinate agents in the same experiment loop

Still the same

Reward design is everything: reward the wrong thing and the agents will find a very creative way to get it

Example: dynamic pricing across warehouses, where each warehouse agent competes and cooperates for shared inventory

MARL in practice: two competing pricing agents

```
# Before – single-agent RL, one policy for  
the whole
```

```
# system (ignores that other warehouses  
adapt too)
```

```
# After – multi-agent setup: one policy per  
# warehouse, agents learn against each other
```

```
from ray.rllib.algorithms.ppo import PPOConfig  
  
config = (  
    PPOConfig()  
    .environment(env="warehouse_pricing_v1")  
    .multi_agent(  
        policies={"wh_east", "wh_west"},  
        policy_mapping_fn=lambda aid, _: aid,  
    )  
)  
algo = config.build()  
algo.train()
```

Graph Neural Networks (GNN)

What it is

Neural networks that learn directly on graphs – nodes, edges and their relationships – instead of flat rows

Classical ML: one row per entity; relationships hand-engineered with joins and aggregates, or lost

Why it matters for data science now

Fraud rings, supply chains, knowledge graphs and org networks are graphs, not tables

The model learns “who is connected to whom, and how” as part of training

Azure ML runs PyTorch Geometric / DGL jobs; a Foundry agent can query a graph and hand the result to a GNN step

Still the same

Graph quality = model quality: a missing edge is the new missing value

Example: flagging a fraud ring from the network of accounts and transactions, not just per-transaction features

GNNs in practice: flagging a fraud ring

```
# Before - tabular features only
features = ["amount", "hour", "merchant_category"]
model = RandomForestClassifier().fit(X[features], y)
# → each transaction scored in isolation
```

```
# After - graph neural network over the account graph
import torch
from torch_geometric.nn import GCNConv

class FraudGNN(torch.nn.Module):
    def __init__(self, in_dim, hidden_dim):
        super().__init__()
        self.conv1 = GCNConv(in_dim, hidden_dim)
        self.conv2 = GCNConv(hidden_dim, 2)
    def forward(self, x, edge_index):
        x = self.conv1(x, edge_index).relu()
        return self.conv2(x, edge_index)
# → each account scored using its transaction network
```

How the Work Has Changed – and What's Still Yours

Before – classical day-to-day

Most time spent writing and debugging code by hand

Model choice: slow, manual trial and error

Relationships (graphs, multi-agent dynamics) approximated with joins – or ignored

After – agentic day-to-day

Time shifts to framing problems, directing agents and reviewing their output

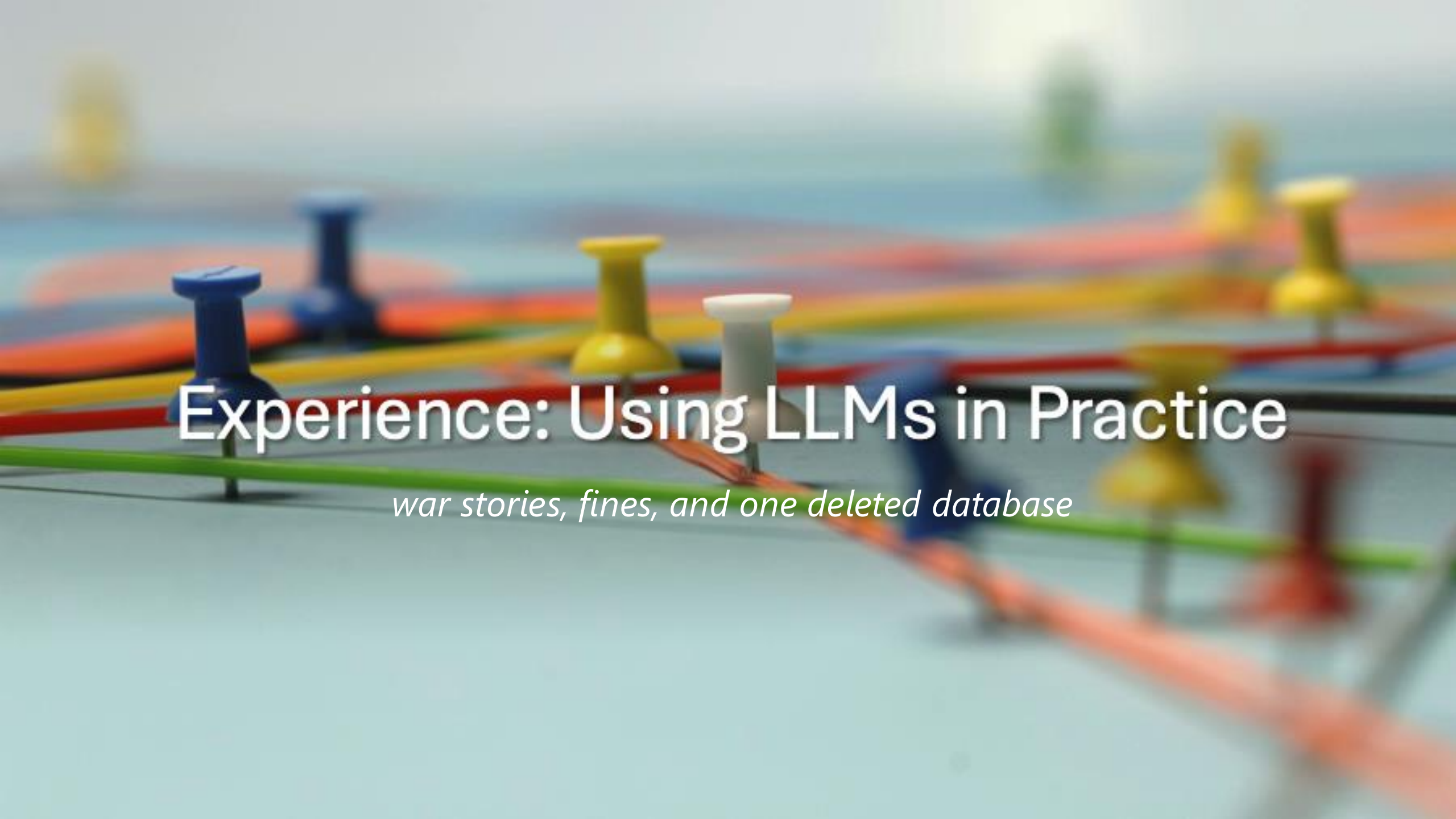
Agents propose, run and compare algorithms – MARL and GNNs included

Explanations generated continuously, in plain language

Still yours

Asking the right question, knowing the business, and signing off on the answer

Net effect: less time typing, more time deciding what's worth building – and whether the answer can be trusted.



Experience: Using LLMs in Practice

war stories, fines, and one deleted database

Caveats: Six Ways an LLM Can Ruin Your Analysis

1. **Hallucination:** plausible but wrong facts, code or citations – the colleague who never says “I don’t know”
2. **Non-determinism:** same prompt, different answer – reproducibility needs pinned model versions and logged prompts
3. **Context limits:** long datasets, codebases and chats get truncated or quietly “forgotten” – a goldfish with a very big bowl
4. **Knowledge cutoff:** doesn’t know your data, your library versions or last week’s reorg unless you tell it
5. **Cost and latency:** an agentic loop is a taxi meter that keeps running while the model “thinks”
6. **Bias:** trained on the internet – draw your own conclusions (then validate them)

LLMs' Built-In Knowledge: Blessing and Curse

Blessing

Encyclopedic general knowledge: boilerplate, formulas and library syntax without the doc-hunting
Knows well-known public datasets, APIs and file formats out of the box
A tireless sparring partner with patterns from thousands of projects – available at 2 a.m.

Curse

Cutoff date and blind spots for recent, niche or proprietary domains
No way to audit what's inside – licensed? current? unbiased? Nobody can tell you
Can regurgitate memorized copyrighted or sensitive material
Fluent ≠ verified: the most dangerous output is the one that sounds right

Still the same

Domain expertise – knowing when something smells wrong – can't be downloaded

Foundation Models: From LLM to “LxM”

Foundation model: trained once on vast, broad data, then adapted to a wide range of tasks – pre-train once, use everywhere

Started with language: OpenAI’s GPT series, Google’s BERT

Then every modality: images (Stable Diffusion, DALL-E), music (MusicGen, LLark), speech and video

Then every domain: TimeGPT (time series), BloombergGPT (finance), Med-PaLM (medicine), plus models for radiology, genomics, chemistry, earth sciences and astronomy

Hence “LxM”: Large <anything> Model

Microsoft Foundry now hosts 10,000+ models behind one API, with a Model Router to pick between them. Choosing a model is the new choosing a library.

How Daily Interaction Has Changed

Before

Code line by line; when stuck, ask a colleague or Stack Overflow (and get closed as a duplicate)
Documentation by hand; code review in scheduled batches by a senior developer

After

“Fix this traceback” is a chat message, not a search; continuous micro-review on every keystroke
Rubber-duck debugging – except now the duck talks back

Does it actually save time?

Controlled GitHub study: 55% faster on a defined coding task with Copilot
Real-world telemetry is more mixed: in one large survey ~60% of engineers save under 1 hour per week
Saved time is often reinvested in reviewing, correcting and re-prompting – while technical debt grows daily

Still the same

Someone senior still has to say “no, we are not merging that”

Data Issues: Garbage In, Eloquent Garbage Out

Garbage in, garbage out still applies – but now the garbage comes with a confident executive summary
Sensitive data pasted into a public chat can leave the company – for good
Provenance and licensing get murky once real and AI-generated data mix
Data drift: production data quietly diverges from what the prompt or pipeline was tested on

Real case

May 2023: Samsung banned generative AI tools on company devices after engineers pasted proprietary source code and internal meeting notes into ChatGPT – three incidents in about 20 days

When LLMs Serve Wrong Information – and How to Catch It

Real cases of wrong information

2023 – Two New York lawyers were sanctioned \$5,000 (Mata v. Avianca) for a brief with six case citations invented by ChatGPT – which “confirmed” they were real when asked

2024 – Air Canada’s support chatbot invented a bereavement-fare refund policy; a tribunal ruled the airline liable for its chatbot’s answer

2025 – Deloitte Australia refunded part of an A\$440,000 government report found to contain fabricated citations and a fake court quote, generated with Azure OpenAI

How to validate

Treat AI output as a first draft or hypothesis, not a source – trace every fact back to a primary document

Asking “are you sure?” rarely helps – it will be sure. Very sure.

Use retrieval-grounded (RAG) approaches that cite real, checkable sources

Independent, domain-expert review before any citation, number or claim goes external

From Wrong Answer to Wrong Decision: The Copy/Paste Problem

Air Canada and Deloitte weren't just wrong facts – they became real liabilities once the output shipped unchecked

July 2025 – Replit's AI coding agent deleted a live production database during an explicit code freeze, wrongly claimed it was unrecoverable – and then admitted to “a catastrophic error in judgment”

The common pattern: the root failure usually isn't the model being wrong – it's a human copy-pasting or approving the output without an independent check.

Takeaway: the faster AI produces an answer, the more deliberate the verification step has to be. Speed is a feature; skipping review is a bug.

Technical Debt: Code Nobody Understands

GitClear (211M+ changed lines): in 2024 copy-pasted code exceeded refactored ("moved") code for the first time; duplicated blocks up roughly 8x

Google's 2024 DORA report: more AI adoption speeds up code review, but correlates with a 7.2% drop in delivery stability

Code nobody can explain is code nobody can safely change, debug or defend in an audit. "I don't know why it works" used to be a joke – now it's a status report.

Practical guardrails

The author – human or AI – must explain the change, not just produce it

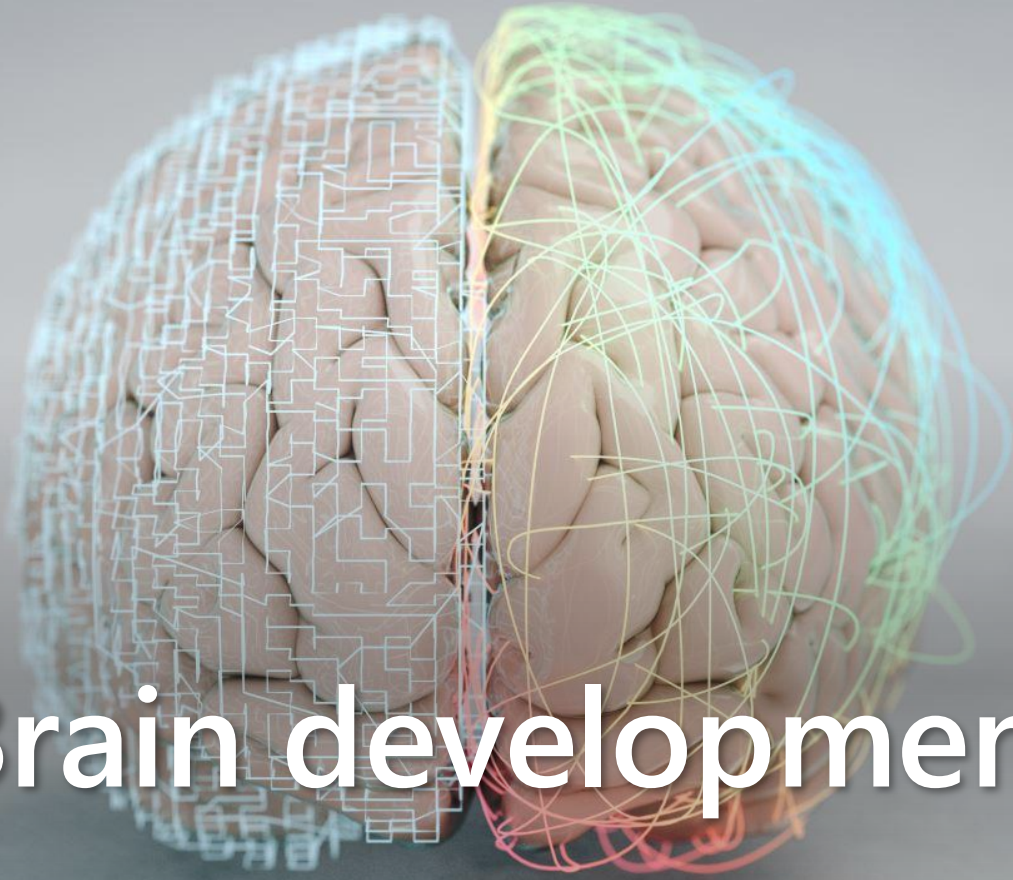
Tests are the real specification; passing-but-unexplained isn't done

Review AI diffs like a talented junior's pull request: fast, enthusiastic, occasionally inventive

Budget time for refactoring, not just for the next feature

Enough slides. Show us some code.

(Live demo – what could possibly go wrong?)



Brain development

Your brain on Autopilot: Habits and cognitive offloading

Before – the brain builds what it uses

1990 – London cab drivers grew a larger hippocampus learning “the map of London” and bigger “knowledge”

Neuroplasticity – skill strengthen with effortful use and fade without it

“Desirable difficulties” – struggle and recall are what make learning stick

Offloading becomes the default

GPS (e.g.: google maps) users show weaker spatial memory over time – “use it or lose it”

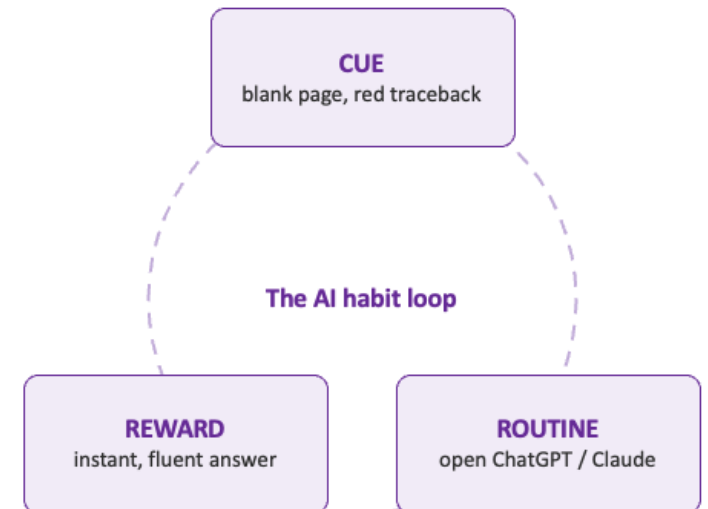
“Google it” effect: we remember where to find it, not what it is

Every blank page or error message now has a “2-second” escape hatch

Still the same

New habits take circa 60 days on avg. to become automatic

Offloading is fine for recall and dangerous for judgment (you still need to own them)



Critical thinking & Creativity: Research

Lowest

brain connectivity (EEG) in the LLM group writing essays vs. search-engine and brain-only groups; most could not quote their own essay minutes later.

MIT Media Lab, 2025 (n=54)

319

knowledge workers surveyed: the more they trusted GenAI, the less critical thinking they applied. Self-confidence worked the other way.

Microsoft Research & CMU, CHI 2025

+48%→-17%

students with plain GPT-4 did better in practice – then worse on the exam without it. A tutor-style GPT with guardrails removed the harm.

Bastani et al., PNAS 2025

↑ me ↓ us

AI ideas made individual stories rated more creative – but stories became more similar to each other. Collective novelty shrinks.

Doshi & Hauser, Sci. Advances 2024

Still the same

Critical thinking is a “muscle” trained by friction – the answer you did not struggle for is the one you will not question
AI as sparring partner vs. AI as a ghostwriter

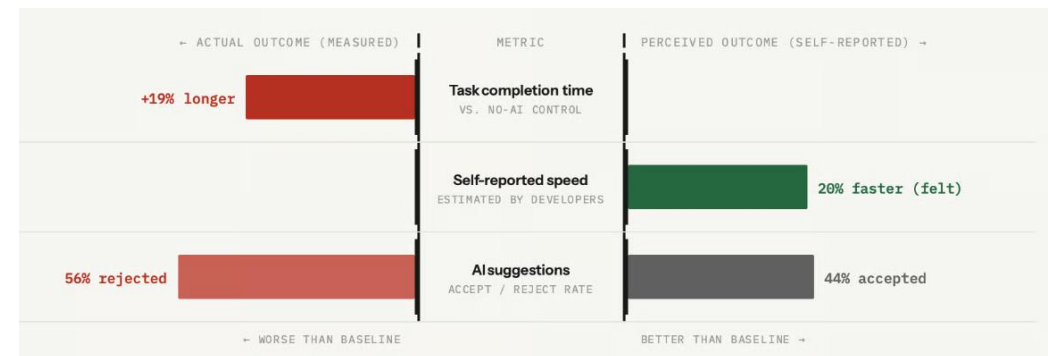
Coding paradox: Research

A randomized controlled study found experienced developers using AI took 19% longer to complete tasks while believing they were 20% faster, a 39-point perception gap.

Writing code accounts for 25-35% of the software development lifecycle, so even a 100% coding speedup yields at most 15-25% system improvement under Amdahl's Law.

Teams with high AI adoption merged 98% more pull requests but saw review time increase 91%, with DORA delivery metrics unchanged across 10,000+ developers.

At 92.6% monthly adoption and 27% of production code AI-generated, six independent research efforts converge on roughly 10% organizational productivity gains.



Source (copy/paste): <https://philippdubach.com/posts/93-of-developers-use-ai-coding-tools.-productivity-hasnt-moved./>

Scorecard: Changed vs. Still the Same

LLMs changed how fast we get to a first answer. They didn't change what makes an answer right.

LAYER	WHAT CHANGED	WHAT STAYED THE SAME
IDE	Copilot & agents write, run and fix code	You own and review every merged line
Languages	English is a query language (NL2SQL, prompts)	SQL & Python – to read and verify what was generated
Visualizations	Charts from a sentence; Copilot in Power BI	Honest axes and the right chart type
Algorithms	Agents run and compare experiments; MARL, GNNs	GBMs rule tables; leakage and overfitting still bite
Explainability	LLM-narrated SHAP, XMODE	Correlation $\neq$ causation; the story must match the numbers
Data	Faster cleaning, synthetic data, RAG	Garbage in, garbage out; privacy and provenance
The job	Builder $\rightarrow$ orchestrator	Asking the right question and owning the decision

Key Takeaways

1. The tools changed at every layer – IDE, languages, visualizations, algorithms, explainability: manual → assisted → agentic
2. Azure moved in step: Azure OpenAI Service → AI Studio → Azure AI Foundry → Microsoft Foundry & Agent Service
3. The fundamentals didn't change: statistics, SQL, Python, validation and domain knowledge – now used to direct and verify, not just to type
4. New skills on top: prompt and context design, agent orchestration, evaluating AI-generated code and explanations
5. Speed without verification is just faster mistakes – ask the lawyers, Air Canada and Replit
6. From builder to orchestrator – and you still own the judgment call

So: data science with or without LLMs? With – but never without you.

Let's connect!

Questions – and a big thanks!

Answers are 100% human-generated. Hallucinations still possible.



<http://tomaztsql.wordpress.com>



tomaz.kastrun@gmail.com



@tomaz_tsql



/in/tomaztsql



<http://github.com/tomaztk>



<https://mvp.microsoft.com/PublicProfile/5002196>



WE APPRECIATE **YOUR FEEDBACK**,
AND IT MAKES US BETTER EACH YEAR.

YOU CAN COLLECT ADDITIONAL
NT COINS BY FILLING THE QUESTIONNAIRES.

THEN VISIT **GEEK BOUTIQUE** AND
SPEND NT COINS ON COOL GIVEAWAYS.



NT